

ZłoteBranieAI

Dokumentacja techniczna

System predykcji warunków meteorologicznych
i aktywności ryb oparty na modelach deep learning

Wersja: 1.0 | Data: 2026-04 | Serwer: AWS EC2

Spis treści

Spis treści

1. Opis projektu
 - 1.1. Modele predykcyjne
 - 1.2. Przepływ danych
2. Użyte technologie i środowisko
 - 2.1. Środowisko sprzętowe
 - 2.2. Język programowania i środowisko
 - 2.3. Biblioteki i frameworki
 - 2.4. Baza danych InfluxDB
 - 2.5. API danych meteorologicznych Open-Meteo
3. Architektura i struktura systemu
 - 3.1. Warstwy systemu
 - 3.2. Struktura plików
 - 3.3. Atomowa podmiana modelu (model_swap.py)
4. Pipeline danych i baza InfluxDB
 - 4.1. Pozyskiwanie danych historycznych
 - 4.2. Struktura InfluxDB
 - 4.3. Pobieranie danych z InfluxDB do treningu
5. Model PatchTST - prognoza pogody 24h
 - 5.1. Architektura modelu
 - 5.2. Konfiguracja modelu
 - 5.3. Inżynieria cech wejściowych
 - 5.4. Trening modelu
 - 5.5. Predykcja i korekcja bias
6. Model TFT - prognoza pogody 10 dni
 - 6.1. Architektura TFT
 - 6.2. Dane wejściowe TFT
 - 6.3. Blending NWP
 - 6.4. Opis wybranych metod - tft_model.py
7. Algorytm predykcji aktywności ryb
 - 7.1. Koncepcja i innowacyjność
 - 7.2. Preferencje gatunków (SPECIES_PREFS)
 - 7.3. Funkcja compute_bite_probability()
 - 7.4. Fine-tuning z Fitzroy Basin
 - 7.5. Trening modeli LightGBM

- 8. REST API
 - 8.1. Uruchomienie serwera
 - 8.2. Obsługa różnych lokalizacji
 - 8.3. Opis endpointów
 - 8.4. Format odpowiedzi
- 9. Testy
 - 9.1. Struktura i filozofia testowania
 - 9.2. Testy jednostkowe (test_unit.py)
 - 9.3. Wyniki testów
 - 9.4. Testy integracyjne i API (test_integration.py)
- 10. Instrukcja obsługi
 - 10.1. Wymagania wstępne
 - 10.2. Instalacja zależności
 - 10.3. Konfiguracja swap
 - 10.4. Inicjalizacja bazy danych
 - 10.5. Trening modeli
 - 10.6. Uruchomienie serwisów
 - 10.7. Weryfikacja działania
 - 10.8. Monitorowanie i logi
- 11. Plan rozwoju i podsumowanie
 - 11.1. Planowane funkcjonalności
 - 11.2. Znane ograniczenia
 - 11.3. Podsumowanie
- 12. Referencje

1. Opis projektu

ZłoteBranieAI to system predykcji warunków wędkarskich oparty na trzech niezależnych modelach uczenia maszynowego. System stanowi komponent większej platformy mobilnej dedykowanej wędkarzom i zarządcom zbiorników wodnych w Polsce. Platforma ta obejmuje aplikację mobilną na systemy iOS i Android, aplikację webową, backend API z autoryzacją JWT oraz planowany system czujników IoT do pomiaru parametrów wody w zbiornikach. Niniejszy moduł predykcji AI integruje się z pozostałymi częściami systemu poprzez REST API, przyjmując koordynaty geograficzne dowolnego zbiornika wodnego i zwracając szczegółowe prognozy meteorologiczne oraz prognozy aktywności ryb.

System odpowiada na realną lukę rynkową, istniejące aplikacje wędkarskie takie jak Fishbrain czy Navionics oferują jedynie ogólne dane historyczne połowów lub proste prognozy pogody, nie stosując modeli głębokiego uczenia do integracji wielowymiarowych danych środowiskowych. ZłoteBranieAI jako pierwsze znane rozwiązanie łączy meteorologię, biologię ryb i rzeczywiste dane hydrobiologiczne z australijskiego zbiornika Fitzroy Basin w jeden zintegrowany pipeline predykcyjny wdrożony na produkcyjnym serwerze.

1.1. Modele predykcyjne

System składa się z trzech głównych modeli predykcyjnych, z których każdy odpowiada za inny aspekt warunków wędkarskich:

Model PatchTST: Generuje godzinową prognozę 9 zmiennych meteorologicznych (temperatura powietrza, temperatura odczuwalna, punkt rosy, prędkość wiatru, porywy wiatru, ciśnienie atmosferyczne, opady, wilgotność względna, zachmurzenie) na horyzont 24 godzin od aktualnego momentu. Model oparty jest na architekturze Patch Time Series Transformer, która tokenizuje szereg czasowy przez podział na patche dobowe, co pozwala na efektywne uczenie wzorców na różnych skalach czasowych.

Model TFT: Generuje dobową prognozę 8 zmiennych meteorologicznych na horyzont 10 dni z trzema kwantylami niepewności — optymistycznym (q90), medianowym (q50) oraz pesymistycznym (q10). Model Temporal Fusion Transformer dodatkowo oblicza prawdopodobieństwo opadów (PoP) dla każdego dnia prognozy. Predykcje TFT są wzmacniane danymi NWP (Numerical Weather Prediction) z Open-Meteo API za pomocą adaptacyjnego blendingu z wagami malejącymi wraz z horyzontem prognozy.

Model LightGBM: Generuje godzinowe prawdopodobieństwo brania ryb (0–100%) dla 11 gatunków: karp, leszcz, szczupak, okoń, sandacz, płoć, lin, sum, kleń, wzdręga oraz amur. Model oparty jest na algorytmie gradientowego wzmacniania drzew decyzyjnych i implementuje oryginalny wieloczynnikowy algorytm biologiczny, który uwzględnia temperaturę wody, ciśnienie atmosferyczne, fazę księżyca, porę dnia i roku, warunki opadowe oraz odkryte wzorce z danych hydrobiologicznych australijskiego zbiornika Fitzroy Basin.

1.2. Przepływ danych

Dane meteorologiczne są pozyskiwane z dwóch źródeł. Dane historyczne (do 5 lat wstecz) pobierane są z Open-Meteo Archive API i przechowywane w lokalnej bazie InfluxDB, co eliminuje konieczność ponownego pobierania przy każdym treningu. Dane bieżące i prognostyczne (ostatnie 7 dni i 16 dni naprzód) pobierane są z Open-Meteo Forecast API bezpośrednio przed każdą predykcją. System automatycznie priorytetyzuje dane z InfluxDB nad API w celu minimalizacji opóźnień i ograniczeń rate limiting.

Wyniki predykcji są zapisywane do bucketu predictions w InfluxDB z tagami identyfikującymi zbiornik wodny (reservoir_id, reservoir_name, owner, lat, lon), co umożliwia filtrowanie i wizualizację danych w narzędziach takich jak Grafana. Całość systemu udostępniana jest przez REST API zaimplementowane w FastAPI, które przyjmuje requesty z aplikacji mobilnej i zwraca skompilowane prognozy wszystkich trzech modeli w jednej odpowiedzi JSON.

2. Użyte technologie i środowisko

2.1. Środowisko sprzętowe

System wdrożony jest na instancji AWS EC2 klasy t3.small wyposażonej w 2 wirtualne rdzenie CPU (Intel Xeon, architektura x86-64), 2 GB RAM oraz 20 GB dysku SSD. Ze względu na ograniczoną ilość pamięci RAM skonfigurowano dodatkowy swap o rozmiarze 2 GB na pliku /swapfile. Serwer działa pod kontrolą systemu Amazon Linux 2 z interpreterem Python 3.9. Brak procesora GPU wymusiło trenowanie wszystkich modeli wyłącznie na CPU, co wpłynęło na dobór rozmiarów architektur, wszystkie modele są celowo małe, aby mieścić się w ograniczeniach sprzętowych.

2.2. Język programowania i środowisko

Cały system zaimplementowany jest w języku Python 3.9. Wybór Pythona jest naturalny w kontekście uczenia maszynowego ze względu na bogaty ekosystem bibliotek ML/DL, dojrzałe narzędzia do przetwarzania danych oraz aktywną społeczność. Do zarządzania zależnościami używany jest pip z plikiem requirements.txt. Kod jest uruchamiany jako procesy tła za pomocą nohup, a logi zapisywane są do katalogu logs/.

2.3. Biblioteki i frameworki

Biblioteka	Wersja	Zastosowanie	Uzasadnienie wyboru
PyTorch	2.8.0	Trenowanie modeli PatchTST	Standard branżowy dla głębokiego uczenia, wsparcie CPU
Hugging Face Transformers	4.x	Architektura PatchTST	Gotowa implementacja PatchTST z konfiguracją
pytorch-forecasting	1.4.0	Architektura TFT	Specjalizowana biblioteka dla prognozowania szeregów
pytorch-lightning	2.6.0	Framework treningowy TFT	Automatyzacja pętli treningowej, checkpointy
LightGBM	latest	Model brań ryb	Szybki, wydajny na CPU, odporny na overfitting
scikit-learn	latest	Skalowanie danych	MinMaxScaler, train_test_split
pandas	latest	Przetwarzanie danych	DataFrames dla szeregów czasowych
numpy	latest	Obliczenia numeryczne	Operacje macierzowe, transformacje kątowe
InfluxDB Client	2.x	Baza danych	Klient Python dla InfluxDB 2.x

Biblioteka	Wersja	Zastosowanie	Uzasadnienie wyboru
FastAPI	latest	REST API	Async, automatyczny Swagger, walidacja Pydantic
Uvicorn	latest	Serwer ASGI	Wydajny serwer dla FastAPI
requests	latest	HTTP klient	Pobieranie danych z Open-Meteo API
joblib	latest	Serializacja skalerów	Zapis/odczyt obiektów sklearn

2.4. Baza danych InfluxDB

InfluxDB 2.x to baza danych szeregów czasowych (TSDB - Time Series Database) zoptymalizowana pod kątem przechowywania i odpytywania danych z regularnie powtarzającymi się pomiarami. W przeciwieństwie do relacyjnych baz danych, InfluxDB przechowuje dane w tzw. measurements (odpowiednik tabel), do których przypisane są tagi (indeksowane metadane, np. nazwa zbiornika) oraz pola (wartości liczbowe, np. temperatura). Dane są automatycznie agregowane i kompresowane w zależności od polityki retencji.

Wybór InfluxDB zamiast np. PostgreSQL wynika z kilku powodów: natywne wsparcie dla danych czasowych z rozdzielczością sekundową, wbudowany język Flux do zaawansowanych zapytań agregujących, wydajne przechowywanie dużych woluminów pomiarów godzinowych (43 000+ rekordów na 5 lat) oraz gotowa integracja z Grafaną do wizualizacji.

2.5. API danych meteorologicznych Open-Meteo

Open-Meteo to darmowe API danych meteorologicznych bez limitu kluczy API, udostępniające dane z analiz atmosferycznych ERA5 oraz modeli numerycznych NWP. System korzysta z dwóch endpointów: Archive API (archive-api.open-meteo.com) dla danych historycznych od 1940 roku do około 5 dni temu oraz Forecast API (api.open-meteo.com) dla danych bieżących i prognoz na 16 dni naprzód. Wybór Open-Meteo zamiast płatnych alternatyw (ECMWF API, AccuWeather) wynika z braku kosztów i wystarczającej jakości danych dla celów projektu.

Kod implementuje mechanizm exponential backoff przy błędach 429 (rate limit) z opóźnieniami 8, 16, 32, 64, 128 i 256 sekund pomiędzy kolejnymi próbami, co zapewnia odporność na chwilowe przeciążenia API.

3. Architektura i struktura systemu

3.1. Warstwy systemu

System zorganizowany jest w cztery warstwy funkcjonalne, które współpracują ze sobą w określonej kolejności:

Warstwa danych: Odpowiada za pozyskiwanie i przechowywanie danych meteorologicznych. Składa się z Open-Meteo Archive API (dane historyczne 5 lat), Open-Meteo Forecast API (dane bieżące i prognozy 16 dni) oraz lokalnej bazy InfluxDB (cache danych historycznych i zapis predykcji). Skrypt `openmeteo_to_influx.py` zarządza backfillem historycznym i codzienną aktualizacją danych.

Warstwa modeli: Trzy niezależne modele predykcyjne: PatchTST (plik `patchtst_model.py`), TFT (plik `tft_model.py`) oraz LightGBM (plik `fish_model.py`). Każdy model ma własny zestaw plików treningowych (`*_train.py`) i predykcyjnych (`*_predict.py`). Modele są trenowane raz w tygodniu przez `scheduler.py` i zastępowane atomowo po sukcesie treningu dzięki mechanizmowi `model_swap.py`.

Warstwa API: FastAPI serwer (`api_server.py`) przyjmuje requesty HTTP z aplikacji mobilnej. Każde zapytanie zawiera koordynaty geograficzne zbiornika (latitude, longitude) oraz opcjonalne metadane (`reservoir_id`, `reservoir_name`, `owner`). Serwer uruchamia modele dla podanej lokalizacji, kompiluje wyniki i zwraca odpowiedź JSON.

Warstwa zapisu: Wyniki predykcji są zapisywane przez `influx_writer.py` do bucketu `predictions` w InfluxDB z tagami identyfikującymi zbiornik. Umożliwia to historyczne śledzenie prognoz i porównanie z rzeczywistymi warunkami w Grafanie.

3.2. Struktura plików

Plik	Rola	Opis
<code>patchtst_model.py</code>	Model PatchTST	Definicja architektury, fetch danych, inżynieria cech, funkcja <code>predict()</code>
<code>patchtst_train.py</code>	Trening PatchTST	Pętla treningowa, skalowanie, early stopping, zapis modelu
<code>patchtst_predict.py</code>	Predykcja 24h	Orchestrator predykcji godzinowej, zapis JSON
<code>tft_model.py</code>	Model TFT	Konfiguracja TFT, fetch danych dziennych, <code>compute_pop()</code>
<code>tft_train.py</code>	Trening TFT	Trening z PyTorch Lightning, walidacja, <code>TrainingContext</code>
<code>tft_predict.py</code>	Predykcja 10d	Predykcja dzienna z blendingiem NWP, zapis JSON

Plik	Rola	Opis
fish_model.py	Model ryb	SPECIES_PREFS, compute_bite_probability(), add_features(), Fitzroy loader
fish_train.py	Trening ryb	Generowanie danych, fine-tuning z Fitzroy, zapis 11 modeli .lgb
fish_predict.py	Predykcja ryb	Predykcja brań 24h dla wszystkich gatunków
api_server.py	REST API	FastAPI endpointy /health i /predict (GET i POST)
scheduler.py	Harmonogram	Cotygodniowy trening, codzienna aktualizacja InfluxDB
openmeteo_to_influx.py	Pipeline danych	Backfill historyczny, daemon aktualizacyjny, estymacja czujników wodnych
influx_writer.py	Zapis predykcji	Funkcje write_weather_24h(), write_weather_10d(), write_fish_24h()
model_swap.py	Podmiana modelu	Atomowa podmiana katalogu modelu po sukcesie treningu
token_manager.py	Autoryzacja JWT	Automatyczne odświeżanie tokenów JWT dla zewnętrznego API aplikacji

3.3. Atomowa podmiana modelu (model_swap.py)

Mechanizm atomowej podmiany modelu zapobiega sytuacji, w której aplikacja użytkownika próbuje wczytać model w trakcie jego zapisywania przez proces treningowy. Implementacja opiera się na klasie TrainingContext będącej context managerem.

Podczas treningu wszystkie pliki modelu (wagi, skalery, checkpointy) są zapisywane do katalogu tymczasowego o nazwie <MODEL_DIR>_tmp. Po pomyślnym zakończeniu treningu katalog tymczasowy jest atomowo przemianowany na aktywny katalog modelu za pomocą operacji os.rename(), która na tym samym systemie plików jest operacją atomową na poziomie systemu operacyjnego. W przypadku błędu podczas treningu katalog tymczasowy jest usuwany, a stary model pozostaje nienaruszony i dostępny dla API.

```
class TrainingContext
```

Context manager zarządzający cyklem życia procesu treningowego. Inicjalizuje katalog tymczasowy przed treningiem i wykonuje atomową podmianę po jego sukcesie.

```
__enter__(self)
```

Usuwa poprzedni katalog tymczasowy, jeśli istnieje (pozostałość po przerwaniu treningu), tworzy nowy katalog tymczasowy i kopiuje do niego istniejące pliki modelu takie jak meta.json, które trening może selektywnie nadpisywać.

```
__exit__(self, exc_type, exc_val, exc_tb)
```

W przypadku sukcesu (exc_type == None) wykonuje atomową podmianę: tworzy backup starego katalogu modelu, przemianowuje katalog tymczasowy na aktywny, a następnie

usuwa backup. W przypadku błędu usuwa katalog tymczasowy bez modyfikowania aktywnego modelu, co gwarantuje, że API zawsze ma dostęp do spójnego modelu.

`atomic_save_file(src_path, dst_path)`

Funkcja pomocnicza do atomowej podmiany pojedynczego pliku. Kopiuje plik do lokalizacji tymczasowej (.tmp), a następnie przemianowuje go na docelową nazwę. Używana do aktualizacji pojedynczych plików bez podmiany całego katalogu.

4. Pipeline danych i baza InfluxDB

4.1. Pozyskiwanie danych historycznych

Skrypt `openmeteo_to_influx.py` odpowiada za pobieranie danych meteorologicznych z Open-Meteo API i zapisywanie ich do InfluxDB. Dane historyczne pobierane są z Archive API w kawałkach rocznych (`CHUNK_DAYS=365`), ponieważ API ma ograniczenia co do maksymalnego zakresu dat w jednym zapytaniu. Pomiędzy każdym rocznym zapytaniem stosowane jest opóźnienie jednej sekundy w celu przestrzegania limitów API.

Skrypt symuluje dane z czujników fizycznych, które nie są jeszcze zainstalowane na zbiornikach. Temperatura wody szacowana jest jako wygładzona temperatura powietrza z opóźnieniem 6 godzin. Poziom wody szacowany jest na podstawie kumulatywnych opadów z ostatnich 72 godzin pomniejszonych o ewapotranspirację. pH wody szacowane jest jako wartość bazowa 7.4 modyfikowana przez opady i temperaturę. Tlen rozpuszczony (DO) szacowany jest ze wzoru Bensona i Krause (1984) jako funkcja temperatury wody.

`fetch_archive_chunk(start: str, end: str) -> pd.DataFrame`

Pobiera dane godzinowe z Open-Meteo Archive API dla podanego zakresu dat. Zwraca DataFrame z kolumnami: `timestamp`, `temperature_c`, `humidity_pct`, `pressure_hpa`, `wind_speed_kmh`, `wind_gust_kmh`, `wind_direction`, `rain_rate_mm`, `cloudcover_pct`, `shortwave_rad`, `direct_rad`, `apparent_temp`, `dewpoint`, `wind_dir_sin`, `wind_dir_cos`. Kierunek wiatru jest konwertowany na reprezentację cykliczną `sin/cos`, co eliminuje problem nieciągłości na granicy $0^{\circ}/360^{\circ}$.

`fetch_recent() -> pd.DataFrame`

Pobiera dane z ostatnich 3 dni oraz prognozy na 2 dni naprzód z Open-Meteo Forecast API. Używa parametrów `past_days=3` i `forecast_days=2`. Wynik jest identyczny strukturalnie z `fetch_archive_chunk()`, co pozwala na bezproblemowe łączenie danych historycznych i bieżących w jeden ciągły szereg czasowy.

`estimate_water_sensors(df: pd.DataFrame) -> pd.DataFrame`

Szacuje wartości sześciu czujników wodnych (temperatura wody, poziom wody, pH, TDS, tlen rozpuszczony, mętność) na podstawie danych meteorologicznych. Każda estymacja oparta jest na odrębnym modelu fizycznym lub statystycznym. Temperatura wody obliczana jest jako ważona średnia wygładzonej temperatury powietrza z 72-godzinny `rolling mean` i 6-godzinny przesunięciem czasowym. Mętność wody obliczana jest przez konwolucję intensywności opadów z funkcją zaniku wykładniczego o czasie połowicznego zaniku 6 godzin.

`compute_derived_features(df: pd.DataFrame) -> pd.DataFrame`

Oblicza cechy pochodne wymagane przez modele ML: pochodne ciśnienia (`d_pressure_3h`, `d_pressure_24h`, `pressure_trend`), `lux` (przeliczenie z promieniowania krótkofalowego), `UV index` (szacowany z promieniowania i pory roku), elewację słońca (obliczana astronomicznie dla każdego `timestampu`), flagi świtu i zmierzchu, cykliczne cechy czasowe (`hour_sin`, `hour_cos`, `doy_sin`, `doy_cos`), fazę księżyca i oświetlenie.

```
write_to_influx(df: pd.DataFrame, client: InfluxDBClient,
batch_size: int = 5000)
```

Zapisuje DataFrame do InfluxDB w batchach po 5000 punktów. Dla każdego wiersza tworzone są trzy punkty InfluxDB: pomiar atmosphere (temperatura powietrza, wiatr, opady, promieniowanie), pomiar water (temperatura wody, pH, tlen) oraz pomiar derived (cechy pochodne dla ML). Każdy punkt tagowany jest lokalizacją (location=torun) i źródłem (source=openmeteo_simulated).

4.2. Struktura InfluxDB

System wykorzystuje dwa buckety w InfluxDB:

Bucket historical_data: Przechowuje dane historyczne meteorologiczne i szacowane dane wodne. Retencja nieograniczona. Zawiera trzy measurements: atmosphere (BME280, wiatr, deszcz, UV - 16 pól), water (DS18B20, pH, TDS, DO, turbidity, poziom - 7 pól) oraz derived (cechy pochodne dla modeli ML - 12 pól). Dane są indeksowane tagiem location=torun, co umożliwia łatwe filtrowanie.

Bucket predictions: Przechowuje wyniki predykcji wszystkich trzech modeli. Retencja 90 dni. Zawiera measurements: forecast_weather_24h (predykcje PatchTST), forecast_weather_10d (predykcje TFT z kwantylami) oraz forecast_fish_24h (predykcje brań ryb). Każdy punkt tagowany jest reservoir_id, reservoir_name, owner, lat, lon, generated_at oraz model, co umożliwia porównanie prognoz z różnych zbiorników i różnych generacji modeli.

4.3. Pobieranie danych z InfluxDB do treningu

Funkcja `_fetch_from_influx()` w plikach `patchtst_model.py` oraz `fish_model.py` odpytuje InfluxDB o dane z ostatnich N lat za pomocą języka Flux. Zapytanie pobiera wszystkie pola z measurement atmosphere, przestawia je z formatu long (jeden rekord na pole) do formatu wide (jeden rekord na timestamp) za pomocą operacji pivot, a następnie sortuje po czasie. Wynikowy DataFrame jest przemianowywany (`temperature_c` → `temperature`, `humidity_pct` → `humidity`) na nazwy kolumn wymagane przez modele ML.

Priorytet InfluxDB przed Open-Meteo API wynika z kilku korzyści: brak ograniczeń rate limiting, lokalny dostęp sieciowy (opóźnienie <1ms), jednorazowe pobranie dużego zakresu danych zamiast wielu małych zapytań rocznych oraz możliwość uzupełnienia danych z czujników IoT gdy te zostaną zainstalowane.

5. Model PatchTST - prognoza pogody 24h

5.1. Architektura modelu

PatchTST (Patch Time Series Transformer) to architektura transformerowa zaprojektowana specjalnie dla jednowymiarowych szeregów czasowych (Nie et al., 2023). Kluczową innowacją w stosunku do klasycznych transformerów jest podział szeregu czasowego na patche (okna czasowe) traktowane jako tokeny wejściowe — analogicznie do tokenizacji tekstu w modelach językowych NLP. Podejście to pozwala modelowi na uczenie się lokalnych wzorców w obrębie każdego patcha (np. dobowy cykl temperatury) oraz wzorców globalnych pomiędzy patchami (np. kilkudniowe trendy ciśnienia).

W konfiguracji użytej w projekcie szereg wejściowy 168 godzin jest dzielony na 7 patchy po 24 godziny każdy (PATCH_LENGTH=24, PATCH_STRIDE=24), co oznacza, że każdy patch odpowiada dokładnie jednemu dniu. Model działa w trybie channel-independence (channel_attention=False), co oznacza, że każda z 22 cech wejściowych jest przetwarzana niezależnie przez ten sam blok transformerowy. Tryb ten zmniejsza liczbę parametrów modelu i ryzyko przeuczenia przy stosunkowo małym zbiorze treningowym.

Wyjście modelu to tensor o kształcie [batch, OUTPUT_HOURS, n_features], z którego wyodrębniamy 9 kanałów odpowiadających zmiennym docelowym (TARGETS). Wybrane kanały to te, których indeksy w FEATURES_IN odpowiadają nazwom z listy TARGETS.

5.2. Konfiguracja modelu

```
build_config(n_features: int) -> PatchTSTConfig
```

Tworzy obiekt konfiguracji PatchTSTConfig z biblioteki Hugging Face Transformers. Parametry architektury: num_input_channels=22 (liczba cech wejściowych), context_length=168 (okno historii 7 dni), patch_length=24 (jeden patch = jeden dzień), patch_stride=24 (bez nakładania patchy), prediction_length=24 (horyzont 24h), d_model=32 (wymiarowość przestrzeni embeddingów), num_attention_heads=4 (główce multi-head attention), num_hidden_layers=2 (liczba bloków transformerowych), ffn_dim=64 (wymiar sieci feed-forward), dropout=0.2, head_dropout=0.3, scaling=std (standaryzacja wejścia), loss=mse (funkcja straty), pre_norm=True, norm_type=batchnorm.

```
build_model(n_features: int) -> PatchTSTForPrediction
```

Inicjalizuje model PatchTSTForPrediction na podstawie konfiguracji z build_config(). Model jest inicjalizowany z losowymi wagami i wymaga trenowania przed użyciem do predykcji. Zwraca obiekt gotowy do przeniesienia na urządzenie obliczeniowe (CPU lub GPU).

```
load_model() -> Tuple[PatchTSTForPrediction, torch.device]
```

Wczytuje wytrenowany model z dysku. Sprawdza czy plik patchtst_weather.pth istnieje w katalogu patchtst_files/. Określa dostępne urządzenie obliczeniowe (CPU lub GPU).

Wczytuje wagi modelu z pliku za pomocą `torch.load()` z parametrem `weights_only=True` dla bezpieczeństwa. Przełącza model w tryb ewaluacji (`model.eval()`) i przenosi na wybrane urządzenie. Rzuca wyjątek `FileNotFoundError` jeśli model nie jest wytrenowany.

5.3. Inżynieria cech wejściowych

Funkcja `add_features()` w `patchtst_model.py` oblicza cechy pochodne niezbędne do trenowania modelu. Poniżej opisano szczegółowo każdą z 22 cech wejściowych:

`hour_sin, hour_cos`

Cykliczna reprezentacja godziny doby obliczana jako $\sin(2\pi \cdot h/24)$ i $\cos(2\pi \cdot h/24)$. Kodowanie cykliczne zamiast liniowego jest konieczne, ponieważ godzina 23 i godzina 0 są sobie bliskie w sensie fizycznym (upływ jednej godziny), ale daleko od siebie w reprezentacji liniowej (różnica 23 jednostek). Dwie wartości `sin` i `cos` razem jednoznacznie identyfikują godzinę doby.

`doy_sin, doy_cos`

Cykliczna reprezentacja dnia roku (day of year) obliczana analogicznie do godziny. Koduje sezonowość — model może rozróżnić lato od zimy i odpowiednio dostosować prognozy temperatury i promieniowania słonecznego.

`d_pressure_3h, d_pressure_24h`

Pochodne ciśnienia atmosferycznego: różnica ciśnienia względem wartości sprzed 3 i 24 godzin. Są to kluczowe cechy predykcyjne dla prognozowania pogody, ponieważ gwałtowne zmiany ciśnienia sygnalizują nadchodzące fronty atmosferyczne. Wartości obliczane są przez metodę `diff(n)` na kolumnie `pressure` z `DataFrame`.

`d_temp_6h`

Zmiana temperatury powietrza w ciągu ostatnich 6 godzin. Pomaga modelowi wykryć trendy grzania lub ochładzania w skali półdobowej, które są istotne przy prognozowaniu temperatury na następne 24 godziny.

`wind_dir_sin, wind_dir_cos`

Cykliczna reprezentacja kierunku wiatru obliczana z kolumny `wind_direction` (stopnie). Analogicznie do godziny, kierunek wiatru wymaga kodowania cyklicznego, wiatr z kierunku 350° i 10° jest podobny (prawie północny), ale ma dużą różnicę liniową.

`temp_daily_max, temp_daily_min`

Maksimum i minimum temperatury powietrza z kroczącego okna 24 godzin obliczane przez `rolling(24).max()` i `rolling(24).min()`. Cechy te dają modelowi kontekst dobowej amplitudy temperatur, co pomaga przy prognozowaniu temperatur nocnych i dziennych.

`solar_heating`

Skumulowane promieniowanie słoneczne z ostatnich 3 godzin obliczane jako `rolling(3).mean()` kolumny `shortwave_radiation`. Cecha ta modeluje bezwładność termiczną atmosfery, szczyt temperatury powietrza następuje z opóźnieniem 2–3 godzin

po szczycie nasłonecznienia (Oke, 1987). `Solar_heating` daje modelowi fizyczny sygnał o zakumulowanej energii cieplnej.

`hour_shifted`

Sinus godziny przesuniętej o 2.5 godziny: $\sin(2\pi \cdot (h+2.5)/24)$. Przesunięcie koryguje fakt, że szczyt temperatury powietrza następuje zwykle około godziny 14:30 lokalnie, a nie o 12:00 jak sugerowałby szczyt nasłonecznienia. Bez tego przesunięcia model musiałby samodzielnie nauczyć się opóźnienia termicznego, co jest trudniejsze przy małym zbiorze treningowym.

5.4. Trening modelu

Skrypt `patchst_train.py` implementuje pełny pipeline treningowy dostosowany do ograniczeń sprzętowych serwera (2 GB RAM, 2 rdzenie CPU).

`class SeqDataset(IterableDataset)`

Implementacja datasetu jako `IterableDataset` zamiast standardowego `Dataset`. Kluczowa różnica polega na tym, że `IterableDataset` generuje sekwencje *on-the-fly* podczas iteracji zamiast przechowywać wszystkie sekwencje w pamięci. Dla 43 000 rekordów i okna $168h+24h$ istnieje około 37 000 sekwencji treningowych. Przechowywanie wszystkich sekwencji jako tablic `numpy` zajęłoby kilka GB RAM. `IterableDataset` przechowuje tylko oryginalne dane (~50 MB) i generuje każdą sekwencję w momencie żądania przez `DataLoader`.

`__iter__(self)`

Generator iterujący po indeksach sekwencji w kolejności losowej (gdy `shuffle=True`) lub sekwencyjnej. Dla każdego indeksu wycinane są odpowiednie kawałki tablic `X` i `Y` i konwertowane na tensory `PyTorch`. Losowe mieszanie jest krytyczne dla trenowania transformerów, ponieważ zapobiega uczeniu się artefaktów wynikających z kolejności danych.

`train(years_back, epochs, batch_size)`

Główna funkcja treningowa. Pobiera dane z `InfluxDB` lub `Open-Meteo API`, dodaje cechy, skaluje dane przez `MinMaxScaler`, dzieli chronologicznie na zbiór treningowy (85%) i walidacyjny (15%), buduje model `PatchTST` i optyimizator `AdamW` z `weight_decay=1e-4`. Pętla treningowa trwa do `EPOCHS` epok lub do wyzwolenia `early stopping` z `patience=80` epok bez poprawy `val_loss`. Co 10 epok obliczany jest `loss` na zbiorze walidacyjnym.

5.5. Predykcja i korekcja bias

`predict(df: pd.DataFrame) -> pd.DataFrame`

Przyjmuje `DataFrame` z co najmniej `INPUT_HOURS` (168) godzinami danych meteorologicznych. Pobiera ostatnie 168 wierszy, skaluje je przez wczytany `MinMaxScaler`, konwertuje na tensor `PyTorch` o kształcie `[1, 168, 22]` i uruchamia `forward pass` modelu. Pobiera wyniki z `prediction_outputs` dla 9 kanałów `TARGETS`, odwraca skalowanie przez skalery wyjściowe. Stosuje korekcję bias: porównuje średnią ostatnich 6h obserwacji ze średnią pierwszych 6h predykcji i dodaje 50% różnicy do wszystkich prognozowanych wartości. Stosuje fizyczne ograniczenia: opady ≥ 0 , wiatr $\geq$

0, wilgotność w $[0, 100]$. Oblicza UV analitycznie na podstawie elewacji słońca i zachmurzenia. Zwraca `DataFrame` z 24 wierszami (jedną godziną każdy) i kolumnami odpowiadającymi TARGETS plus timestamp, uv_index i pop_pct.

6. Model TFT - prognoza pogody 10 dni

6.1. Architektura TFT

Temporal Fusion Transformer (TFT) to architektura zaprojektowana przez Google DeepMind (Lim et al., 2021), zoptymalizowana dla wielowariantowych prognoz z kwantylami i interpretowalną uwagą. Implementacja korzysta z biblioteki pytorch-forecasting w wersji 1.4.0. TFT różni się od PatchTST pod kilkoma istotnymi aspektami: przetwarza dane o różnych typach (zmienne znane z przyszłości vs obserwowane tylko w przeszłości), generuje probabilistyczne prognozy z kwantylami zamiast wartości punktowych oraz posiada mechanizm Variable Selection Networks (VSN), który uczy się ważności poszczególnych cech wejściowych.

Architektura TFT składa się z czterech głównych komponentów przetwarzanych sekwencyjnie. Variable Selection Networks wybierają i ważą cechy wejściowe niezależnie dla każdego timestampu, co pozwala modelowi ignorować szumliwe cechy. LSTM Encoder przetwarza ważone cechy historyczne w reprezentację kontekstową. Multi-Head Attention stosuje mechanizm attention na sekwencji historycznej, identyfikując które przeszłe momenty są najbardziej informatywne dla danej prognozy. Quantile Output generuje trzy kwantyle dla każdego z 8 targetów i 10 dni prognozy.

6.2. Dane wejściowe TFT

TFT obsługuje trzy typy zmiennych wejściowych o różnych właściwościach dostępności:

Zmienne docelowe (TARGETS): Osiem zmiennych meteorologicznych w agregacji dziennej: temp_max (maksimum temperatury), temp_min (minimum temperatury), precip_sum (suma opadów), wind_max (maksimum prędkości wiatru), wind_mean (średnia prędkości wiatru), pressure_mean (średnie ciśnienie), cloudcover_mean (średnie zachmurzenie), humidity_mean (średnia wilgotność). Zmienne te są obserwowane do dnia bieżącego i prognozowane na 10 dni naprzód.

Zmienne obserwowane w przeszłości (OBSERVED_PAST): Dodatkowe cechy dostępne tylko dla danych historycznych: temp_mean (średnia temperatura), d_pressure_3d (zmiana ciśnienia w 3 dni), precip_7d_sum (suma opadów z 7 dni), temp_anomaly (anomalia temperatury względem 30-dniowej średniej), nwp_error_temp (historyczny błąd NWP dla temperatury). Te zmienne pomagają modelowi zrozumieć kontekst historyczny, ale nie są dostępne dla okresu prognozy.

Zmienne znane z przyszłości (KNOWN_FUTURE_REALS): Cechy dostępne zarówno dla historii jak i okresu prognozy: nwp_temp_max, nwp_temp_min, nwp_precip, nwp_wind_max, nwp_pressure, nwp_cloudcover (wartości NWP z Open-

Meteo dla całego okna predykcji), `day_sin`, `day_cos`, `month_sin`, `month_cos` (cykliczne cechy kalendarzowe). Zmienne NWP dostarczają modelowi informacji o przewidywanych warunkach atmosferycznych, co znacznie poprawia jakość prognoz szczególnie dla krótkich horyzontów.

6.3. Blending NWP

Blending NWP to technika łączenia prognoz modelu TFT z danymi z modeli numerycznych (NWP) z Open-Meteo. Motywacją jest fakt, że modele numeryczne są bardzo precyzyjne dla krótkich horyzontów (1-3 dni), ale ich jakość spada szybko dla dłuższych horyzontów (7-10 dni), gdzie modele uczenia maszynowego uczące się na danych historycznych mogą być konkurencyjne.

Dla każdego dnia prognozy i każdej zmiennej docelowej stosowana jest formuła blendingu: $q50_final = w \times NWP + (1-w) \times TFT_q50$, gdzie w jest wagą NWP malejącą wykładniczo z horyzontem. Przedziały niepewności ($q10$ i $q90$) są zachowane z TFT i skalowane wokół nowej mediany blendowanej: $q10_final = q50_final - |TFT_q50 - TFT_q10|$.

Wagi NWP są zróżnicowane w zależności od zmiennej docelowej: dla temperatury wynoszą od 92% (dzień 1) do 42% (dzień 10), dla opadów od 95% do 55%, dla wiatru od 85% do 32%. Wyższe wagi dla opadów wynikają z faktu, że modele numeryczne są historycznie lepsze w prognozowaniu opadów niż temperatury dla krótkich horyzontów.

6.4. Opis wybranych metod - `tft_model.py`

```
_fetch_daily_forecast_with_nwp(past_days, forecast_days) -> pd.DataFrame
```

Pobiera dane dzienne z Open-Meteo Forecast API zawierające zarówno dane historyczne jak i prognozy NWP. Zapytanie do API pobiera zmienne dzienne (`temperature_2m_max`, `temperature_2m_min`, `precipitation_sum`, `windspeed_10m_max`, `winddirection_10m_dominant`, `pressure_msl_mean`, `cloudcover_mean`, `relative_humidity_2m_mean`) oraz zmienne godzinowe do obliczenia agregacji dobowych. Funkcja `_build_df()` konwertuje surowe odpowiedzi API na DataFrame z odpowiednimi nazwami kolumn i oblicza cechy pochodne takie jak `temp_mean` jako średnia arytmetyczna `temp_max` i `temp_min`.

```
add_features(df: pd.DataFrame) -> pd.DataFrame
```

Dodaje cechy pochodne wymagane przez TFT: `d_pressure_3d` (zmiana ciśnienia w 3 dni), `precip_7d_sum` (suma opadów z 7 dni), `temp_anomaly` (odchylenie od 30-dniowej średniej kroczącej), `nwp_error_temp` (różnica między wartością NWP a obserwowaną temperaturą dla dni historycznych, zero dla dni przyszłych), `day_sin`, `day_cos`, `month_sin`, `month_cos` (cykliczne kodowanie dnia i miesiąca).

```
compute_pop(conditions: dict) -> float
```

Oblicza prawdopodobieństwo opadów (PoP — Probability of Precipitation) na podstawie czterech warunków: `precip_sum` (suma opadów z prognozy), `humidity_mean` (średnia

wilgotność), `cloudcover_mean` (średnie zachmurzenie) oraz `nwp_precip` (opady z NWP). Algorytm startuje od zera i dodaje punkty proporcjonalnie do intensywności każdego z warunków. Wilgotność powyżej 70% i zachmurzenie powyżej 60% dodają dodatkowe punkty. Wynik jest przycinany do zakresu 0-100.

7. Algorytm predykcji aktywności ryb

7.1. Koncepcja i innowacyjność

Algorytm predykcji brań ryb jest najistotniejszym wkładem naukowym niniejszego projektu. Dostępne rozwiązania komercyjne i badawcze opierają się głównie na danych historycznych połowów zebranych od użytkowników lub prostych regułach ekspertowych (np. ryby biorą przy zmianie ciśnienia). Zaproponowany algorytm idzie dalej w trzech kluczowych kierunkach.

Po pierwsze, implementuje wieloczynnikową funkcję biologiczną opartą na literaturze naukowej z zakresu ichtiologii i ekologii ryb, która jawnie modeluje siedem niezależnych czynników środowiskowych i ich interakcje. Po drugie, odkrywa i implementuje dwa nowe wzorce aktywności ryb wykryte przez analizę statystyczną rzeczywistych danych hydrobiologicznych z australijskiego zbiornika Fitzroy Basin — wzorzec post-deszczowy i wzorzec termiczny. Po trzecie, stosuje technikę transfer learningu między ekosystemami przez fine-tuning modelu LightGBM na znormalizowanych danych australijskich, co wzbogaca syntetyczne dane treningowe o rzeczywiste obserwacje.

7.2. Preferencje gatunków (SPECIES_PREFS)

Każdy z 11 obsługiwanych gatunków ma zdefiniowany słownik preferencji środowiskowych SPECIES_PREFS oparty na literaturze ichtiologicznej (Leatherland i Woo, 2010) i wiedzy ekspertów. Poniżej opisano parametry dla wybranych gatunków:

Gatunek	Optimum wody [°C]	Minimum [°C]	Maksimum [°C]	Pref. ciśnienia	Okno aktywności
Karp	18-25	10	30	stabilne	5:00-21:00
Leszcz	15-22	8	28	stabilne	4:00-10:00
Szczupak	10-18	4	24	spadające	6:00-20:00
Okoń	16-22	8	26	dowolne	6:00-20:00
Sandacz	14-20	6	24	spadające	18:00-23:00
Płoc	16-24	6	28	stabilne	6:00-20:00
Lin	18-26	10	30	stabilne	5:00-11:00
Sum	20-28	12	32	spadające	20:00-6:00
Kleń	12-20	5	25	dowolne	6:00-20:00
Wzdreęga	14-22	6	26	stabilne	7:00-19:00
Amur	20-28	14	32	stabilne	8:00-20:00

7.3. Funkcja compute_bite_probability()

Funkcja `compute_bite_probability(row, species)` oblicza prawdopodobieństwo brania jako sumę ważoną siedmiu czynników biologicznych. Wynik startuje od wartości bazowej 0.5 (50%) i jest modyfikowany addytywnie przez każdy czynnik:

Czynnik 1: Temperatura wody

Jeśli temperatura wody (`water_temp_c`) leży poniżej `water_temp_min` lub powyżej `water_temp_max` dla danego gatunku, funkcja natychmiast zwraca losową wartość z zakresu 0-5%, ponieważ ryby w takich warunkach są praktycznie nieaktywne fizjologicznie. Jeśli temperatura leży w optimum [`opt_lo`, `opt_hi`], score zwiększany jest o +0.20. Jeśli temperatura leży poza optimum ale w dopuszczalnym zakresie, score zmniejszany jest o 0.02 za każdy stopień odchylenia od najbliższej granicy optimum.

Czynnik 2: Wzorzec Fitzroy Basin - spadek temperatury wody

Czynnik innowacyjny wynikający z analizy danych australijskich. Jeśli zmiana temperatury wody w ciągu ostatnich 24 godzin (`water_temp_delta`) jest ujemna i przekracza -0.5°C , score zwiększany jest o `temp_drop_bonus` zdefiniowany w `SPECIES_PREFS` (od +0.05 dla lina do +0.20 dla szczupaka). Mechanizm biologiczny: schłodzenie wody zwiększa zawartość tlenu rozpuszczonego (Benson i Krause, 1984), co stymuluje aktywność metaboliczną ryb. Gwałtowne ocieplenie wody ($\text{delta} > +1.5^{\circ}\text{C}$) skutkuje zmniejszeniem score o -0.10, ponieważ nagła zmiana termiczna może wywoływać szok termiczny.

Czynnik 3: Wzorzec Fitzroy Basin - aktywność po deszczu

Drugi czynnik innowacyjny z analizy danych australijskich. Jeśli suma opadów z ostatnich 24 godzin (`rain_24h_sum`) przekracza 3 mm, ale bieżące opady (`rain_rate_mm`) są poniżej 0.5 mm/h (stan po deszczu, nie podczas), score zwiększany jest o `post_rain_bonus` (od +0.05 do +0.20 zależnie od gatunku). Mechanizm biologiczny: spływ opadowy dostarcza do zbiornika organiczne substancje odżywcze i zmienia mętność wody, co zachęca ryby do żerowania przy zwiększonym zamaskowaniu wizualnym. Ulewa (`rain_rate_mm` > 5 mm/h) skutkuje zmniejszeniem score o -0.30 dla wszystkich gatunków, ponieważ intensywne opady zaburzają warstwowanie termiczne wody.

Czynnik 4: Ciśnienie atmosferyczne

Ryby posiadają pęcherz pławny będący wrażliwym detektorem ciśnienia (Pavlov et al., 2000). Gatunki drapieżne (szczupak, sandacz, sum) z preferencją `pressure_pref=falling` uzyskują +0.20 przy spadku ciśnienia powyżej 1.5 hPa/3h (sygnał nadchodzącego frontu atmosferycznego aktywizującego łańcuch pokarmowy) i -0.20 przy wzroście ciśnienia powyżej 1.5 hPa/3h. Gatunki spokojne (karp, leszcz, lin) z preferencją `pressure_pref=stable` uzyskują +0.15 przy zmianie ciśnienia poniżej 1 hPa/3h. Gwałtowna zmiana ciśnienia powyżej 4 hPa/3h skutkuje -0.25 dla wszystkich gatunków.

Czynnik 5: Pora dnia

Każdy gatunek ma zdefiniowane okno aktywności (`active_hours`) jako para (`h_lo`, `h_hi`). Gatunki nocne jak sum mają `h_lo > h_hi`, co jest obsługiwane specjalnie: warunek aktywności to `hour >= h_lo or hour <= h_hi`. Pobyt w oknie aktywności skutkuje +0.10, a poza nim -0.20. Godzina pobierana jest z pola `timestamp` wiersza `DataFrame`.

Czynnik 6: Świt i zmierzch

Świt i zmierzch to biologicznie wyróżnione pory dobowego cyklu żerowania większości gatunków ryb (Leatherland i Woo, 2010). Flagi `is_dawn` i `is_dusk` obliczane są na podstawie elewacji słońca: `dawn` to elewacja z przedziału $(-6^{\circ}, +6^{\circ})$ w godzinach porannych, `dusk` analogicznie w godzinach popołudniowych. Obecność flagi skutkuje dodaniem `dawn_dusk_bonus` specyficznego dla gatunku (od $+0.15$ dla wzdregi do $+0.50$ dla sandacza).

Czynnik 7: Księżyc

Faza księżyca wpływa na aktywność nocną ryb przez poziom oświetlenia wody (Penteriani et al., 2011). Gatunki preferujące pełnię (`moon_pref=full`) jak karp i lin uzyskują $+0.10$ przy oświetleniu księżyca powyżej 75% (`illum > 75`). Gatunki preferujące now (`moon_pref=new`) jak leszcz, sandacz i sum uzyskują $+0.10$ przy `illum < 25%`, ponieważ ciemność zmniejsza widoczność ryb dla drapieżników i ośmieli je do żerowania. Faza księżyca obliczana jest analitycznie względem znanych dat nowiu.

Czynnik 8: Wiatr

Silny wiatr utrudnia żerowanie ryb przez falowanie powierzchni wody i zaburzenie warstw termicznych. Każdy gatunek ma zdefiniowaną maksymalną tolerowaną prędkość wiatru (`wind_max`). Jeśli `wind_speed_kmh` przekracza `wind_max`, `score` zmniejszany jest proporcjonalnie: $-0.20 \times (\text{wind} - \text{wind_max}) / 10$. Wzdrega jest najbardziej wrażliwa (`wind_max=12 km/h`), sum jest najmniej wrażliwy (`wind_max=35 km/h`).

Po zsumowaniu wszystkich czynników dodawany jest szum gaussowski z odchyleniem standardowym 0.07 (7%) w celu zapobiegania zbyt idealnym wartościom prognostycznym. Wynik jest przycinany do zakresu $[0, 100]$ i zaokrąglany do jednego miejsca po przecinku.

7.4. Fine-tuning z Fitzroy Basin

```
load_fitzroy_features(csv_path: str) -> pd.DataFrame
```

Wczytuje plik CSV z danymi australijskiego zbiornika Fitzroy Basin zawierający 235 dziennych pomiarów: Rainfall (mm), Level (m), E Conductivity ($\mu\text{S/cm}$), Water Temp ($^{\circ}\text{C}$), pH, Turbidity (NTU), Number of fish. Normalizuje temperaturę wody z zakresu australijskiego $[21-29^{\circ}\text{C}]$ do zakresu polskiego $[13-22^{\circ}\text{C}]$ przez skalowanie liniowe, ponieważ gatunki australijskie i polskie mają podobne preferencje termiczne, ale w innych bezwzględnych zakresach. Konwertuje liczbę złowionych ryb na `bite_probability` przez normalizację do 95. percentyla. Dla brakujących cech meteorologicznych (brak danych o ciśnieniu, wietrze) wstawia typowe wartości domyślne. Zwraca `DataFrame` z kolumnami zgodnymi z `FEATURE_COLS` wymaganymi przez `LightGBM`.

7.5. Trening modeli LightGBM

```
train(df, species, fitzroy_df=None) -> lgb.Booster
```

Trening modelu gradientowego wzmacniania drzew dla pojedynczego gatunku. Filtruje dane do wierszy z danym gatunkiem, dzieli na zbiór treningowy (85%) i walidacyjny (15%) chronologicznie. Jeśli `fitzroy_df` jest podany, dane Fitzroy są dodawane do zbioru treningowego z trzykrotnym powtórzeniem (waga $3\times$), co sprawia, że fine-tuning ma znaczący wpływ na model. Parametry modelu: `learning_rate=0.03`, `num_leaves=31`, `feature_fraction=0.8`, `bagging_fraction=0.8`, `reg_alpha=0.1`, `reg_lambda=0.2`. Trening

kończy się po 500 iteracjach lub przy early stopping po 50 rundach bez poprawy RMSE na zbiorze walidacyjnym.

```
generate_training_data(years_back: int = 3) -> pd.DataFrame
```

Pobiera dane meteorologiczne z Open-Meteo Archive API za N lat i generuje syntetyczne bite_probability dla każdego gatunku i każdej godziny. Dla każdego gatunku tworzona jest kopia całego DataFrame z dodaną kolumną species (nazwa gatunku) oraz bite_probability (obliczona przez compute_bite_probability()). Zwraca DataFrame zawierający $N \times 8760 \times \text{liczba_gatunków}$ rekordów, dla 3 lat i 11 gatunków to około 290 000 wierszy.

8. REST API

8.1. Uruchomienie serwera

Serwer API zaimplementowany jest w FastAPI z serwerem ASGI Uvicorn. FastAPI automatycznie generuje interaktywną dokumentację Swagger dostępną pod adresem <http://localhost:8080/docs>, co znacznie ułatwia testowanie i integrację przez programistów aplikacji mobilnej. Wybór FastAPI zamiast Flask wynika z natywnej obsługi async/await, automatycznej walidacji parametrów przez Pydantic oraz wydajniejszego parsowania JSON przez orjson.

Serwer uruchamiany jest z jednym workerem (workers=1), ponieważ serwer ma 2 rdzenie CPU i każde zapytanie predykcyjne może zużywać do 1 rdzenia przez kilka sekund. Używanie wielu workerów przy ograniczonym RAM spowodowałoby konflikty pamięci podczas ładowania modeli PyTorch.

8.2. Obsługa różnych lokalizacji

System obsługuje prognozy dla dowolnych koordynatów geograficznych przez mechanizm dynamicznego nadpisywania zmiennych LAT/LON w modułach modeli. Przed każdą predykcją funkcje `_run_patchtst()` i `_run_tft()` w `api_server.py` tymczasowo nadpisują LAT i LON w importowanych modułach (`patchtst_model.LAT = lat`), wykonują predykcję pobierając dane dla nowej lokalizacji, a następnie przywracają oryginalne wartości w bloku finally. Podejście to jest proste i skuteczne dla pojedynczego wątkowego serwera.

8.3. Opis endpointów

```
GET /health
```

Endpoint diagnostyczny sprawdzający dostępność serwera i modeli. Odpytuje system plików w poszukiwaniu plików modeli (`patchtst_weather.pth`, `tft_weather*.ckpt`, `model_*.lgb`) i zwraca ich status (`available: true/false`) oraz czas ostatniej modyfikacji (`trained_at`). Nie wykonuje żadnych obliczeń ML. Czas odpowiedzi poniżej 1 sekundy.

```
GET /predict
```

Generuje prognozę dla lokalizacji podanej w parametrach query string. Wymagane parametry: `latitude (float)`, `longitude (float)`. Opcjonalne: `reservoir_id (int)`,

reservoir_name (str), owner (str), models (str, domyślnie 'patchtst,tft,fish'). Parametr models pozwala na selektywne uruchomienie modeli: wartość 'fish' uruchomi tylko predykcję ryb (czas ~1s), 'patchtst,fish' uruchomi predykcję pogody i ryb (czas ~5-7s), 'patchtst,tft,fish' uruchomi wszystkie modele (czas ~10-20s).

POST /predict

Identyczna funkcjonalność jak GET /predict, ale parametry przekazywane są w ciele żądania jako obiekt JSON z polami: latitude, longitude, reservoir_id, reservoir_name, owner, models. Zalecany do użycia przez aplikację mobilną, gdy przekazywanych jest wiele parametrów lub nazwy zbiorników zawierają znaki specjalne.

`_predict(lat, lon, reservoir_id, reservoir_name, owner, models_str)`

Wewnętrzna funkcja wykonująca właściwą logikę predykcji. Parsuje parametr models_str na listę modeli do uruchomienia. Sekwencyjnie uruchamia _run_patchtst(), _run_tft() i _run_fish() z obsługą wyjątków — błąd jednego modelu nie przerywa wykonania pozostałych. Zapisuje wyniki do InfluxDB z tagami zbiornika przez _write_to_influx_with_tags(). Mierzy czas wykonania i dodaje pole elapsed_sec do odpowiedzi.

`_write_to_influx_with_tags(result_dict, measurement, reservoir_id, ...)`

Zapisuje wyniki predykcji do InfluxDB z pełnymi tagami lokalizacyjnymi. Dla każdego wiersza prognozy tworzy punkt InfluxDB z tagami: reservoir_id, reservoir_name, owner, lat, lon, generated_at, model. Pola prognozy są zapisywane jako wartości float — dla TFT każde pole kwantylowe (np. temp_max_q10, temp_max_median, temp_max_q90) jest osobnym polem. Zapis wykonywany jest synchronicznie w celu zapewnienia spójności danych przed zwróceniem odpowiedzi HTTP.

8.4. Format odpowiedzi

Odpowiedź z endpointu /predict zawiera następujące pola najwyższego poziomu: generated_at (czas generowania w formacie ISO 8601), latitude i longitude (koordynaty z żądania), reservoir_id i reservoir_name (metadane zbiornika), models_run (lista modeli które uruchomiły się pomyślnie), errors (lista błędów modeli które nie uruchomiły się), elapsed_sec (czas generowania w sekundach), forecast_24h (prognoza PatchTST), forecast_10d (prognoza TFT z kwantylami) i forecast_fish (prognoza LightGBM z best_hours).

Pole forecast_fish zawiera dodatkowo słownik best_hours z najlepszą godziną połowu dla każdego gatunku — jest to godzina z maksymalnym prawdopodobieństwem brania w prognozie 24h wraz z wartością tego prawdopodobieństwa.

9. Testy

9.1. Struktura i filozofia testowania

Zestaw testów zorganizowany jest w dwóch plikach w katalogu tests/: test_unit.py zawierający testy jednostkowe z mockami zewnętrznych zależności oraz

test_integration.py zawierający testy integracyjne i testy API. Plik tests/conftest.py dodaje katalog nadrzędny do sys.path, co pozwala na import modułów projektu bez instalacji pakietu. Plik pytest.ini w katalogu głównym konfiguruje domyślne opcje uruchamiania testów.

Filozofia testowania opiera się na kilku zasadach: testy jednostkowe powinny być izolowane i szybkie (nie wykonują requestów HTTP, nie potrzebują modeli ML), testy integracyjne powinny weryfikować rzeczywiste zachowanie systemu z prawdziwymi danymi, a testy API powinny weryfikować kontrakt interfejsu niezależnie od implementacji. Każdy test powinien testować jeden konkretny aspekt zachowania systemu.

9.2. Testy jednostkowe (test_unit.py)

Plik test_unit.py zawiera 72 testy jednostkowe podzielone na klasy testowe odpowiadające poszczególnym modułom systemu. Funkcje pomocnicze make_weather_df() i make_fish_df() generują syntetyczne DataFrames z losowymi, ale fizycznie sensownymi wartościami, używanymi jako dane wejściowe dla testowanych funkcji.

Klasa testowa	Moduł	Liczba testów	Testowany aspekt
TestPatchTSTFeatures	patchtst_model.py	11	add_features(): zakresy sin/cos, brak NaN, wszystkie cechy
TestPatchTSTModel	patchtst_model.py	4	Architektura: liczba parametrów, kształt tensorów, brak NaN
TestPatchTSTClean	patchtst_model.py	2	_clean(): wypełnianie NaN, zachowanie długości DataFrame
TestFishAddFeatures	fish_model.py	10	add_features(): cechy, faza księżyca, świt/zmierzch, water_temp_delta
TestBiteProbability	fish_model.py	5	compute_bite_probability(): zakres 0-100, nocny sum, zimno
TestMoonPhase	fish_model.py	4	moon_phase(): zakres, znany now księżyca 2000-01-06
TestSolarElevation	fish_model.py	3	solar_elevation(): południe > północ, ujemna elewacja nocna

Klasa testowa	Moduł	Liczba testów	Testowany aspekt
TestFitzroyLoader	fish_model.py	2	load_fitzroy_features(): kolumny, zakres bite_probability
TestWeatherModelFeatures	weather_model.py	3	add_features(), _fix_influx_cols()
TestTFTModelFeatures	tft_model.py	2	compute_pop(): zakres, suche vs mokre warunki
TestInfluxWriter	influx_writer.py	3	Parsowanie timestamp, błąd połączenia bez wyjątku
TestTrainingContext	model_swap.py	3	Tworzenie tmp, podmiana, rollback przy błędzie
TestOpenMeteoToInflux	openmeteo_to_influx.py	3	estimate_water_sensors(): pH, DO, kolumny wyjściowe

Dodatkowo zestaw zawiera 27 testów parametrycznych: 10 testów bite_probability dla każdego gatunku (weryfikacja zakresu 0-100 przy losowych warunkach), 3 testy długości features dla różnych okien wejściowych (168h, 200h, 500h) oraz 4 testy ekstremalnych temperatur (karp i leszcz przy -10°C, szczupak przy 35°C, wzdrenga przy 40°C oczekiwana jest niska aktywność).

9.3. Wyniki testów

Wszystkie testy zostały uruchomione na serwerze AWS EC2 z wytrenowanymi modelami. Wyniki z dnia 2026-04-16:

```
67 passed, 5 skipped, 0 failed — 7.43s
```

5 testów pominiętych (skipped) wynika z dwóch przyczyn: testy TestFitzroyLoader są pomijane, gdy plik Fitzroy_Basin.csv nie jest dostępny w katalogu testów (plik znajduje się w katalogu głównym projektu), a testy TestInfluxWriter są pomijane, gdy moduł influx_writer nie jest dostępny w sys.path podczas uruchamiania testów z katalogu tests/.

9.4. Testy integracyjne i API (test_integration.py)

Plik test_integration.py zawiera testy integracyjne podzielone na trzy grupy: TestPatchTSTPipeline (testy z mockami Open-Meteo dla pełnego pipeline PatchTST), TestRealOpenMeteoIntegration (testy z prawdziwym API, oznaczone @pytest.mark.slow) oraz grupy testów API oznaczone @pytest.mark.api.

Testy API weryfikują: poprawny kod HTTP 200 dla /health i /predict, kod 422 dla brakujących parametrów latitude/longitude, poprawną strukturę odpowiedzi JSON (pola models_run, errors, elapsed_sec), 24 godziny w forecast_24h, 10 dni w forecast_10d, poprawny zakres prawdopodobieństw ryb (0-100%), kolejność kwantyli TFT ($q_{10} \leq \text{median} \leq q_{90}$), czas odpowiedzi fish <10s, czas health <1s, oraz 5 kolejnych requestów bez błędów.

10. Instrukcja obsługi

10.1. Wymagania wstępne

Przed uruchomieniem systemu należy spełnić następujące wymagania: system operacyjny Linux (Amazon Linux 2 lub Ubuntu 20+), Python 3.9 lub nowszy, minimum 2 GB RAM plus skonfigurowany swap 2 GB, minimum 2 rdzenie CPU, minimum 10 GB wolnego miejsca na dysku, działająca instancja InfluxDB 2.x z utworzonymi bucketami historical_data i predictions, dostęp do internetu dla pobierania danych z Open-Meteo API.

10.2. Instalacja zależności

Wszystkie zależności Python instalowane są przez pip. Zalecane jest użycie wirtualnego środowiska Python aby uniknąć konfliktów z innymi projektami:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install transformers pytorch-forecasting pytorch-lightning
pip install lightgbm scikit-learn pandas numpy joblib
pip install influxdb-client requests fastapi uvicorn
pip install pytest pytest-mock
```

10.3. Konfiguracja swap

Konfiguracja swap jest niezbędna na serwerze z 2 GB RAM, ponieważ trening PatchTST wymaga do 1.5 GB RAM a system operacyjny i inne procesy zajmują pozostałą pamięć. Brak swap spowoduje zabicie procesu treningowego przez OOM killer.

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

10.4. Inicjalizacja bazy danych

Przed pierwszym treningiem należy wypełnić InfluxDB danymi historycznymi. Operacja backfillu 5 lat danych trwa około 15-20 minut ze względu na ograniczenia rate limiting Open-Meteo API i pobieranie danych rok po roku:

```
python openmeteo_to_influx.py --verify
python openmeteo_to_influx.py --backfill --years 5
```

10.5. Trening modeli

Trening wszystkich trzech modeli powinien być uruchamiany sekwencyjnie, a nie równolegle, ze względu na ograniczenia RAM. Każdy model jest trenowany jako proces tła z przekierowaniem logów. Trening PatchTST trwa 3-6 godzin, TFT 2-4 godziny, a Fish tylko około 5 minut:

```
mkdir -p logs patchtst_files tft_model_files fish_model_files
nohup python -u patchtst_train.py > logs/patchtst_train.log 2>&1 &
# Poczekaj na zakończenie PatchTST przed uruchomieniem TFT
nohup python -u tft_train.py > logs/tft_train.log 2>&1 &
nohup python -u fish_train.py --fitzroy Fitzroy_Basin.csv >
logs/fish_train.log 2>&1 &
```

Postęp treningu można śledzić przez podgląd pliku logu:

```
tail -f logs/patchtst_train.log
```

10.6. Uruchomienie serwisów

Po zakończeniu treningu należy uruchomić trzy procesy tła: serwer API, scheduler treningów i daemon aktualizacji danych:

```
nohup python -u api_server.py --port 8080 > logs/api_server.log 2>&1 &
nohup python -u scheduler.py > logs/scheduler.log 2>&1 &
nohup python -u openmeteo_to_influx.py --daemon > logs/influx_daemon.log
2>&1 &
```

10.7. Weryfikacja działania

Po uruchomieniu serwisów należy zweryfikować poprawność działania systemu:

```
# Sprawdzenie stanu API
curl http://localhost:8080/health
```

```
# Testowe zapytanie predykcyjne
curl
"http://localhost:8080/predict?latitude=53.0138&longitude=18.5981&models
=patchtst,fish"
```

```
# Uruchomienie testów jednostkowych
pytest tests/test_unit.py -v
```

10.8. Monitorowanie i logi

Wszystkie procesy zapisują logi do katalogu logs/. Pliki logów to: api_server.log (requesty HTTP, błędy predykcji), scheduler.log (harmonogram treningów), patchtst_train.log / tft_train.log / fish_train.log (postęp treningu, RMSE, early stopping), influx_daemon.log (aktualizacje danych). Logi są dopisywane do istniejących plików przy restarcie procesów.

Aby sprawdzić, czy procesy działają:

```
cat logs/scheduler.pid | xargs ps -p 2>/dev/null || echo 'DEAD'  
cat logs/patchtst_train.pid | xargs ps -p 2>/dev/null || echo 'DEAD'
```

11. Plan rozwoju i podsumowanie

11.1. Planowane funkcjonalności

System w obecnej formie stanowi działający prototyp produkcyjny, który może być rozbudowywany w kilku kierunkach:

Czujniki IoT: Planowane jest wdrożenie boi pomiarowych, których prototyp został już opracowany. Instalacja fizycznych czujników, takich jak pH, temperatury DS18B20, mętności czy poziomu wody pozwoi zastąpić obecne estymaty rzeczywistymi danymi z terenu. Bezpośrednie pomiary znacząco podniosą jakość predykcji brań, szczególnie w oparciu o wzorzec Fitzroy oraz termikę wody.

Dane 850 hPa: Uzupełnienie danych treningowych o ciśnienie i temperaturę na poziomie izobarycznym 850 hPa z bazy reanaliz ERA5 (Copernicus Climate Data Store). Dane te są kluczowe dla modelowania adwekcji mas powietrza i frontów atmosferycznych, co powinno poprawić prognozowanie gwałtownych zmian pogody przez model PatchTST.

Dane od użytkowników: Zbieranie rzeczywistych danych o połowach od użytkowników aplikacji mobilnej (gatunek, waga, godzina, lokalizacja) i użycie ich do fine-tuningu modelu LightGBM. Zastąpi to syntetyczne dane biologiczne rzeczywistymi obserwacjami specyficznymi dla polskich zbiorników i gatunków.

Rozszerzenie gatunków: Dodanie kolejnych gatunków ryb popularnych w Polsce: troć wędrowną, lipień europejski, certa, jazgarz, ukleja, karaś. Każdy gatunek wymaga zdefiniowania SPECIES_PREFS na podstawie literatury ichthyologicznej.

Optymalizacja wydajności: Implementacja cache odpowiedzi API z TTL jednej godziny po stronie serwera. Prognoza meteorologiczna zmienia się co godzinę, więc wielokrotne odpytywanie tego samego zbiornika w ciągu godziny może zwracać cached wyniki bez uruchamiania modeli ML.

11.2. Znane ograniczenia

System posiada kilka znanych ograniczeń wynikających z przyjętych kompromisów:

- Model PatchTST osiąga RMSE temperatury $\sim 2.56^{\circ}\text{C}$, wartość powyżej docelowej 2.0°C . Wynika to z braku danych 850 hPa i relatywnie małego zbioru treningowego (5 lat = 43 000 rekordów). Retrening z danymi ERA5 powinien obniżyć RMSE.
- Temperatura wody jest szacowana z danych atmosferycznych zamiast mierzona fizycznie. Estymacja oparta na rolling mean temperatury powietrza jest przybliżeniem, które może się znacząco różnić od rzeczywistości w głębokich zbiornikach zaporowych.

- Model LightGBM trenowany jest na danych syntetycznych generowanych przez funkcję biologiczną, a nie na rzeczywistych obserwacjach połowów. Wzorce z Fitzroy Basin dotyczą australijskiego zbiornika i gatunków tropikalnych, które różnią się biologicznie od polskich gatunków.
- Serwer z 2 rdzeniami CPU i 2 GB RAM obsługuje tylko ~10 równoległych requestów (patchtst+fish) przy success rate $\geq 95\%$. Większe obciążenie wymaga modernizacji infrastruktury lub implementacji kolejkowania.

11.3. Podsumowanie

W ramach niniejszej pracy zaprojektowano i zaimplementowano system predykcji warunków wędkarskich składający się z modelu PatchTST do godzinowej prognozy pogody, modelu TFT do 10-dniowej prognozy z kwantylami niepewności oraz oryginalnego algorytmu predykcji brań ryb opartego na LightGBM. System wdrożono na serwerze produkcyjnym AWS EC2 i udostępniono przez REST API integrujące się z aplikacją mobilną.

Kluczowym wkładem naukowym jest wieloczynnikowy algorytm predykcji aktywności ryb, który po raz pierwszy w literaturze łączy następujące elementy: formalna specyfikacja preferencji biologicznych 11 gatunków polskich ryb w formie parametrycznych słowników, odkrycie i implementacja wzorca post-deszczowego i wzorca termicznego przez analizę statystyczną danych hydrobiologicznych, transfer wiedzy między ekosystemami australijskim i polskim przez normalizację temperatur i fine-tuning modelu LightGBM, oraz fizycznie uzasadnione modelowanie 8 czynników środowiskowych z odwołaniami do literatury ichtiologicznej.

Zestaw 72 testów jednostkowych i integracyjnych (67 passed, 5 skipped) weryfikuje poprawność implementacji wszystkich kluczowych komponentów systemu. Testy obciążeniowe wykazały, że system obsługuje do 10 równoległych requestów ze success rate $\geq 95\%$ przy medianie czasu odpowiedzi poniżej 7 sekund dla kompletnej prognozy pogody i ryb.

12. Referencje

- [1] Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR 2023.
<https://arxiv.org/abs/2211.14730>
- [2] Oke, T.R. (1987). Boundary Layer Climates (2nd ed.). Routledge.
(bezwładność termiczna atmosfery)
- [3] Lim, B., Arik, S.Ö., Loeff, N., Pfister, T. (2021). Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting. International Journal of Forecasting, 37(4), 1748-1764.
<https://arxiv.org/abs/1912.09363>
- [4] pytorch-forecasting — biblioteka do prognozowania szeregów czasowych.
<https://pytorch-forecasting.readthedocs.io/>
- [5] Hugging Face Transformers — implementacja PatchTST.
https://huggingface.co/docs/transformers/model_doc/patchtst

- [6] Leatherland, J.F., Woo, P.T.K. (eds.) (2010). Fish Diseases and Disorders, Vol. 2: Non-infectious Disorders (2nd ed.). CABI. (biologia aktywności ryb)
- [7] Benson, B.B., Krause, D. (1984). The concentration and isotopic fractionation of oxygen dissolved in freshwater and seawater in equilibrium with the atmosphere. *Limnology and Oceanography*, 29(3), 620-632.
- [8] Pavlov, D.S., Lupandin, A.I., Skorobogatov, M.A. (2000). The Effects of Flow Turbulence on the Behavior and Distribution of Fish. *Journal of Ichthyology*, 40(S2).
- [9] Penteriani, V., Kulagnanam, A., del Mar Delgado, M. et al. (2011). Bright moonlight triggers natal dispersal departures. *Behavioral Ecology and Sociobiology*, 65, 1367-1374.
- [10] Ke, G., Meng, Q., Finley, T. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
<https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [11] Open-Meteo — otwarte API danych meteorologicznych. <https://open-meteo.com/>
- [12] InfluxDB 2.x dokumentacja. <https://docs.influxdata.com/influxdb/v2/>
- [13] FastAPI dokumentacja. <https://fastapi.tiangolo.com/>